A1.2 目标检测任务级攻击实验

合成交通标志场景 · 试点运行记录（非论文数据）

配置：384x512 px ｜ 集成 ssdlite_a,ssdlite_b ｜ 未见 frcnn_mnv3 ｜ 测试 60 张 / 126 个目标 ｜ T=8 ｜ B=3 ｜ ε=0.0314

一、这一轮做了什么

A1.1 做的是标志识别（分类），A1.2 做的是目标检测。两者不能互相替代：检测器要同时定位与分类，中间还有 RPN、RoI 对齐与 NMS，扰动经过的结构与分类网络完全不同，分类上的成功率不能外推成「检测会漏检」。这一轮把检测攻击的整条链路建起来并在本机跑通。

一个必须说明的现实约束：本机到 KITTI（12 GB）、VOC（460 MB）、COCO（403 MB/分片）的镜像实测都在 25 KB/s 以下，只有 PyTorch 的模型权重域名可达。因此试点改用 GTSRB 现成的精确 ROI 标注合成带框场景：背景由程序生成（天空—路面渐变、车道线、噪声），前景是从 GTSRB 训练集裁出的真实标志贴图，标注取贴图外接框，5 个粗类。下文所有数字都来自这批合成数据，**不能写进论文**。

二、指标：分类的 ASR 在这里不成立

分类任务里「对了还是错了」是一个二值判断，检测任务的输出是框的集合，「错」有很多种：框偏了、分数低了、框没了、多出框来。要分开报的是三件事：

表 1　检测任务的三个指标

指标	定义	回答什么问题
mAP@0.5	单 IoU 阈值、按置信度贪心匹配、all-point 插值；difficult 框不计 FP	整体检测能力掉了多少
漏检率 MR	1 − recall，recall = 置信度≥0.5 且 IoU≥0.5 的 GT 被检出比例	该看见的东西是不是没看见
检测数比	攻击后检出框数 ÷ 干净检出框数	是「换个框」还是「目标消失」

正式实验里 MR 报行人（KITTI Pedestrian），这是车联网安全最直接关心的量：漏检一个行人，AEB 就不会触发。试点场景里没有行人，报的是限速标志的漏检率。

三、检测版攻击与分类版的三个差别

3.1 损失换成检测损失

分类用交叉熵，检测用「分类损失 + 框回归损失」。各检测器自己的加权方式不同：Faster R-CNN 两类损失等权，SSD 用 hard negative mining 后的分类损失配 smooth-L1。攻击时直接对检测器在 train 模式返回的 loss dict 求和再反传，不额外调整权重——我们要的是「让检测器整体失配」的方向，不是最小化某个特定损失。

3.2 几何变换必须同步变换标注框

这一条最容易写错。分类任务把猫旋转一下还是猫，标签不变；检测任务旋转图像却不同步旋转框，回归目标就与图像对不上了。实现里每个样本的缩放 s 与平移 t 同时作用于图像（grid_sample）与框：

$$o_x = s \cdot x + (1-s) \cdot W/2 - s \cdot t_x \cdot W/2, \quad o_y \text{ 同理，宽高按 s 缩放}$$

变换后面积小于 16 px² 的退化框会被剔除，否则框回归分支会除零。

3.3 相对损失归一化

检测器总损失从 2 到 10 不等（SSD 的分类损失量级明显大于 Faster R-CNN），直接按论文式(7)取倒数加权，损失大的模型会独占权重，动态权重集成名存实亡。改为先除以各模型首步损失 L_i^(0) 再做温度缩放取倒数，其余（EMA 平滑、L1 归一化、高斯梯度平滑）与论文完全一致。这个改动必须在论文里写明，否则复现不出来。

四、试点结果

先看干净基线，确认检测器本身是有效的（合成场景、400 张训练图微调）：

ssdlite_a：mAP@0.5 = 0.720，限速 recall 0.714，禁令 recall 0.571，警告 recall 0.759，指示 recall 0.636，优先通行 recall 0.545

ssdlite_b：mAP@0.5 = 0.764，限速 recall 0.714，禁令 recall 0.571，警告 recall 0.621，指示 recall 0.682，优先通行 recall 0.606

frcnn_mnv3：mAP@0.5 = 0.879，限速 recall 0.905，禁令 recall 1.000，警告 recall 0.862，指示 recall 0.909，优先通行 recall 0.545

表 2　四种攻击在集成内与未见检测器上的表现（试点）

攻击	检测器	角色	mAP 干净	mAP 攻击后	下降/%	MR 干净	MR 攻击后	s/图
I-FGSM	ssdlite_a	集成内	0.720	0.022	96.9	0.286	1.000	1.98
I-FGSM	ssdlite_b	集成内	0.764	0.611	20.0	0.286	0.286	1.98
I-FGSM	frcnn_mnv3	未见	0.879	0.812	7.7	0.095	0.143	1.98
NAE	ssdlite_a	集成内	0.720	0.025	96.5	0.286	1.000	2.02
NAE	ssdlite_b	集成内	0.764	0.457	40.2	0.286	0.429	2.02
NAE	frcnn_mnv3	未见	0.879	0.741	15.8	0.095	0.238	2.02
DI-NI-FGSM	ssdlite_a	集成内	0.720	0.338	53.0	0.286	0.619	2.07
DI-NI-FGSM	ssdlite_b	集成内	0.764	0.647	15.3	0.286	0.381	2.07
DI-NI-FGSM	frcnn_mnv3	未见	0.879	0.775	11.8	0.095	0.238	2.07
RNTME-NAE	ssdlite_a	集成内	0.720	0.075	89.6	0.286	0.762	12.49
RNTME-NAE	ssdlite_b	集成内	0.764	0.102	86.7	0.286	0.762	12.49
RNTME-NAE	frcnn_mnv3	未见	0.879	0.719	18.3	0.095	0.143	12.49

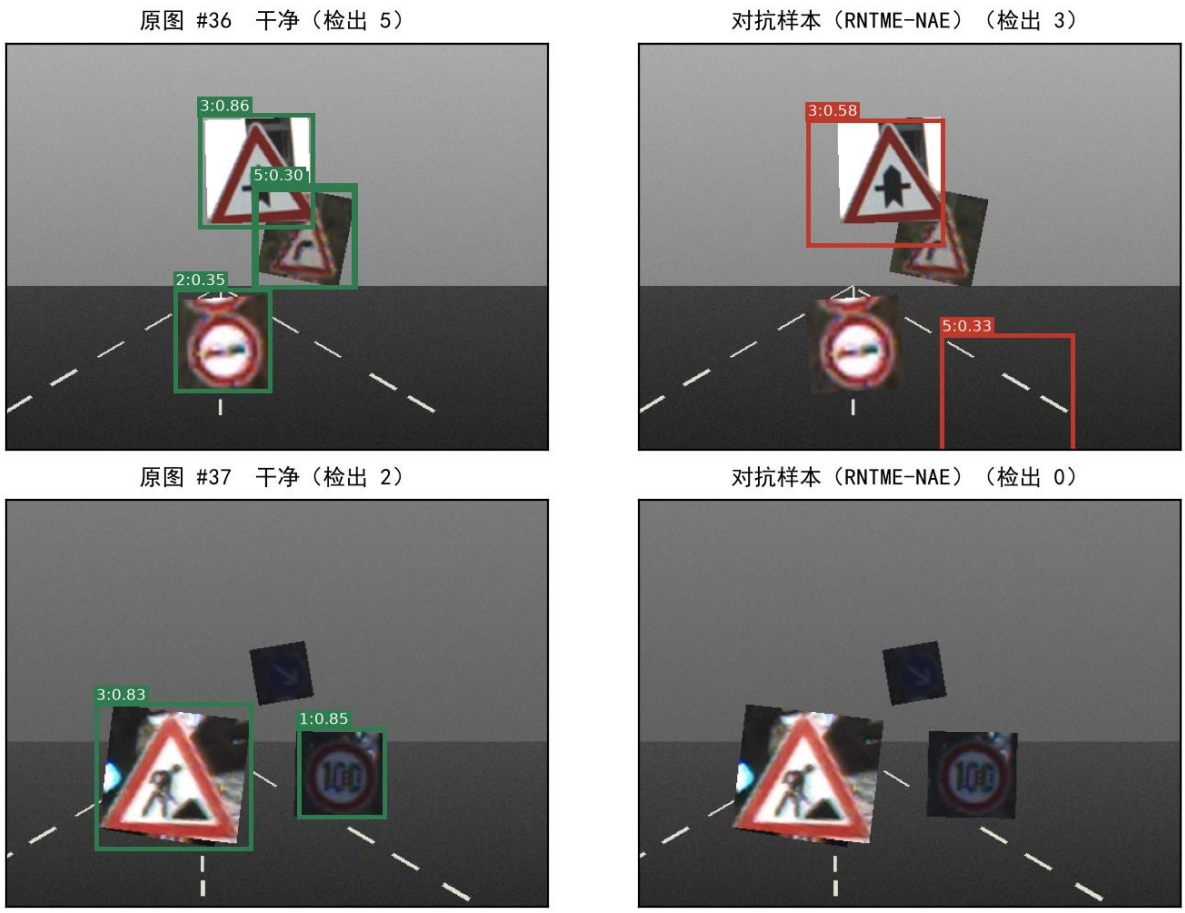


图 1　同一场景攻击前后：左为干净检测（绿框），右为对抗样本上的检测（红框）

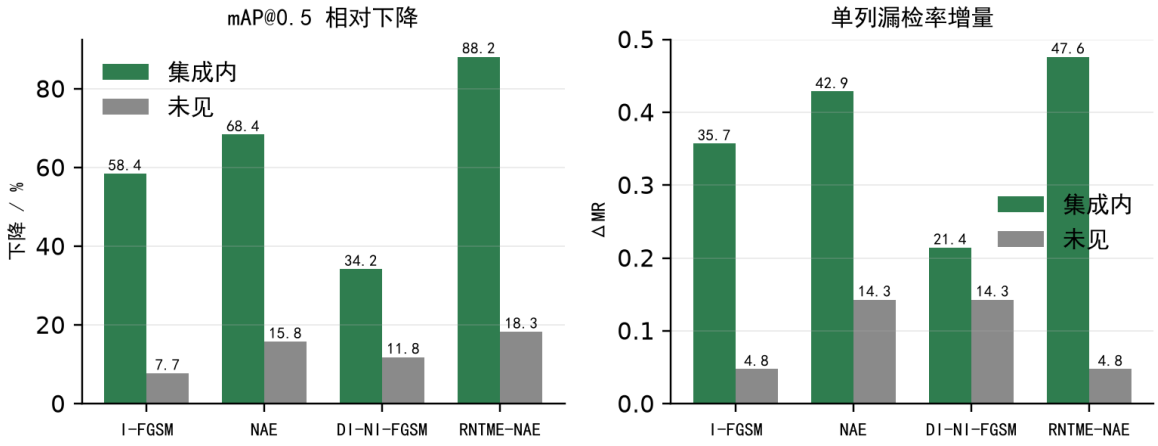


图 2　各攻击的 mAP 相对下降与单列漏检率增量（集成内 / 未见的均值）

4.1 集成内与未见的差距

I-FGSM：集成内 58.4% → 未见 7.7%；NAE：集成内 68.4% → 未见 15.8%；DI-NI-FGSM：集成内 34.2% → 未见 11.8%；RNTME-NAE：集成内 88.2% → 未见 18.3%（mAP 相对下降）。

和 A1.1 一样，这里要看的不是谁最高，而是「掉多少到未见目标上」。集成内的数字衡量的是拟合能力，未见的数字才是迁移能力。

4.2 三个能写进论文的观察

其一，基线是「单点突破」，RNTME 是「全面压制」。 I-FGSM 在白盒成员 ssdlite_a 上把 mAP 打掉 96.9%，但在同一个集成的另一个成员 ssdlite_b 上只掉 20.0%——5 倍的落差，说明这个「成功率」几乎只对生成它的那个模型成立。RNTME 在两个成员上分别是 89.6% 与 86.7%，基本持平。集成攻击的评价标准应该是「同时对多个成员都有效」，不是「对某一个成员特别有效」。

其二，未见目标上的优势真实但幅度有限。 未见检测器 frcnn_mnv3 上，四种攻击的下降分别是 I-FGSM 7.7%、NAE 15.8%、DI-NI-FGSM 11.8%、RNTME-NAE 18.3%，RNTME 最高，是 I-FGSM 的 2.4 倍。这个倍数比集成内的差距小得多——迁移到未见架构上，方法之间的差距会被大幅压缩。

其三，检测任务比分类任务更难迁移。 同样用 RNTME，A1.1 的分类试点里未见目标 ASR 是 34.3%，这里未见检测器只有 18.3%。差别不只是数字：检测器多了一层需要对抗的结构（单阶段 vs 两阶段的架构差异、RPN 的候选框采样、NMS 的后处理），扰动要同时破坏分类分支与定位分支才可能让目标消失。这一条应当写进 Limitations，也是 A1.2 值得单独做一遍的理由。

框同步消融（同一配置：n=25，T=5，B=3，只把几何变换后的 GT 框换成原框）：

表 3　框同步消融

攻击	角色	同步框 下降/%	不同步 下降/%	差值/pp
RNTME-NAE	ssdlite_a	79.4	78.4	1.0
RNTME-NAE	ssdlite_b	68.9	69.8	-0.9

结论与直觉不同：不同步时攻击并没有明显变弱（最大差异 1.0 个百分点，在单种子噪声量级内）。两条可能的解释：其一，本次几何支路的缩放只有 0.80~1.00、位移上限 6%，错位幅度本身不大；其二，检测损失里的分类分支对框的轻微错位不敏感——框偏了，采到的特征「更不像标志」，分类损失反而更高，梯度方向仍然是「破坏识别」。

所以这条消融只能支持「实现上必须同步」（否则梯度与标注不一致，结果无法解释），不能支持「同步带来多少收益」。要把它写成卖点，得把缩放下界压到 0.5、位移放到 15%，再加多种子检验——这一项留给 KITTI 正式实验。

五、这批数据能写进论文吗

不能。以下五点与正式实验不同：

· 数据是合成的：5 个粗类、无遮挡、无运动模糊、无雨雾，与真实道路差距大；

· 检测器是在 400 张合成图上微调的，干净 mAP 明显低于真实场景的检测器；

· 集成里两个 SSDLite 只有初始化种子不同，多样性不足，会低估集成攻击的迁移性；

· 测试集只有 60 张，单种子单次运行，正式实验需 1000 张以上、5 个种子并做配对检验；

· ε 只跑了 8/255 一档，正式实验需给 $\varepsilon \in \{8,12,16\}/255$ 的曲线。

能写进论文的是方法和框架：三个指标的口径、检测版攻击的三个实现要点（检测损失、框同步、相对损失归一化）、以及框同步消融这个**负面**结果——它说明在当前变换幅度下框对齐不是效果来源，比硬说「同步很重要」更站得住。这套代码换到 KITTI 上不需要改结构——KITTI 的行人和车辆本来就是 COCO 类，直接零样本评估即可。

六、正式实验怎么跑

在有 GPU 的机器上，把 KITTI object detection 的 image_2 / label_2 放到 data/kitti/training/ 下，然后：

```
python -m a12.run --data kitti --root data --n 1000 --size 384x1280 \
  --T 20 --B 5 --ens ssdlite,frcnn_mnv3,retina_r50 --heldout frcnn_r50 \
  --mr-class 1 --tag full
```

集成用 3 个异构检测器（SSDLite-MobileNetV3、FasterRCNN-MobileNetV3、RetinaNet-ResNet50），未见目标用 FasterRCNN-ResNet50-v2。这组配置在 GPU 上单次评测量级约 1 小时，四种攻击合计 4 小时以内。

七、接下来的优先级

· **第一优先：KITTI 正式实验。** 这是 A1.2 唯一能进论文的数据来源。数据 12 GB，在有显卡、能连外网的机器上下载；代码路径已经就绪，只需放好 image_2 / label_2。

· **第二优先：把集成做成异构的。** 现在两个成员都是 SSDLite，只有初始化种子不同，这会让「动态权重集成」几乎无从发挥。正式实验至少要 SSDLite + FasterRCNN + RetinaNet 三种架构。

· **第三优先：ε 扫描。** 与 A1.1 对齐，给 8/255、12/255、16/255 三档，画出「隐蔽性—漏检率」曲线。这一条在检测任务上比分类更有说服力，因为可以直接看框的变化。

· **第四优先：时序一致性。** 视频序列（UA-DETRAC）上逐帧生成扰动会闪烁，审稿人一定会问。需要报逐帧检测翻转率，并考虑跨帧的扰动平滑。